

SNOMED tokenization via semantic coverage

Yuan-Yuan & Alexis

August 26, 2026

1 Objective

Let $\mathcal{V}$ denote the set of concepts in the SNOMED CT graph ($|\mathcal{V}| \simeq 400\,000$), and let $\mathcal{M} \subseteq \mathcal{V}$ be the subset of mapped hospital concepts ($|\mathcal{M}| \simeq 40\,000$). The objective is to select

$$\mathcal{T} \subseteq \mathcal{V}, \quad |\mathcal{T}| = k,$$

called the set of *tokens*, in order to represent the mapped concepts using a smaller vocabulary.

Main hypothesis. A concept is well represented when its semantic branches reaches tokens. Then, the semantic loss corresponds to branches that do not reach any token before a depth budget D . In future work, we can also consider losses related to the logic of SNOMED, for example primitive concepts (see Section 8).

2 SNOMED graph and notation

Let

$$G = (\mathcal{V}, \mathcal{E})$$

be the directed multigraph of SNOMED. A relationship is a triple

$$e = (u, r, v) \in \mathcal{E},$$

where $u \in \mathcal{V}$ is the source concept, $r \in \mathcal{R}$ is the relationship type, and $v \in \mathcal{V}$ is the destination concept.

For SNOMED concepts, the terms *child* and *parent* are reserved for the ISA hierarchy. For

$$u \xrightarrow{\text{ISA}} v,$$

u is more specific than v : u is a child and v is a parent. For $r \neq \text{ISA}$, we refer only to source u , destination v , or the value of attribute r .

Define the sets of outgoing and incoming edge occurrences by

$$\text{Out}(u) = \{(u, r, v) \in \mathcal{E}\}, \quad d^+(u) = |\text{Out}(u)|,$$

$$\text{In}(v) = \{(u, r, v) \in \mathcal{E}\}.$$

$\text{Out}(u)$ and $\text{In}(v)$ keep the full triple, to make the direction and type of each relationship immediately explicit in this document. In the code, no need to store u in $\text{Out}(u)$ and v in $\text{In}(v)$.

3 Tokenized representation using context trees

Once $\mathcal{T}$ is fixed, the representation of a mapped concept is a context tree. A context is like a parenthesis that records that some tokens were found under the same occurrence of a non-ISA relationship.

The rule is as follows. ISA relationships are transparent: they continue the search inside the current context. Non-ISA relationships create subcontexts: they introduce a new semantic aspect. When a token is reached, it is attached to the current context.

For example,

`finding site(kidney, laterality(left))`

records that `left` qualifies the kidney site.

3.1 Tokenization algorithm

Tokenizing a mapped concept $c \in \mathcal{M}$ follow the recursive procedure **EXPAND**. It starts with the call

$$\text{EXPAND}(c, q_0, 0),$$

where q_0 is the initial context. More generally, a call

$$\text{EXPAND}(u, q, d)$$

means that:

- u is the current SNOMED concept, reached from c after exactly d relationships;
- q is the current context, determined by the non-ISA relationship occurrences encountered on the path from c to u ;
- d is the total depth from the initial concept c .

Thus, the initial recursive state is $(u, q, d) = (c, q_0, 0)$.

At each call:

1. if $u \in \mathcal{T}$, the token for this branch is found: attach u to context q and stop this branch;
2. otherwise, if $d = D$ or $\text{Out}(u) = \emptyset$, declare the branch uncovered;
3. otherwise, expand every outgoing relationship of u :
 - (a) an ISA relationship preserves the current context q ;
 - (b) a relationship $r \neq \text{ISA}$ creates a fresh subcontext attached to q and labelled by r .

Two distinct occurrences of the same non-ISA relationship create two distinct contexts. The order of the first two cases matters: a token located exactly at depth D is accepted.

Listing 1: Tokenization of a mapped concept

```

TOKENIZE(c, T, D):
    q0 = new_context(parent_context=None, relation=ROOT)
    EXPAND(c, q0, 0)
    return q0

EXPAND(u, q, d):
    if u is in T:
        attach token u to context q
        return

    if d == D or Out(u) is empty:
        mark this branch as uncovered in context q
        return

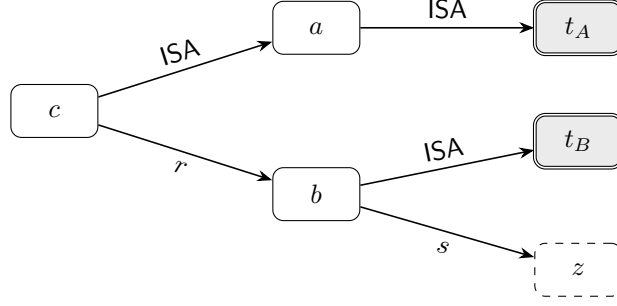
    for each edge (u, r, v) in Out(u):
        if r == ISA:
            EXPAND(v, q, d + 1)
        else:
            q_new = new_context(parent_context=q, relation=r)
            EXPAND(v, q_new, d + 1)

```

If $c \in \mathcal{T}$, the initial call immediately returns $\text{ROOT} \rightarrow c$.

3.2 Step-by-step tokenization example

Let $D = 2$, let $\mathcal{T} = \{t_A, t_B\}$, and consider the following graph. Double-bordered vertices are tokens; z is a non-token leaf.



The execution is as follows:

1. `TOKENIZE(c, T, 2)` creates the root context q_0 and calls `EXPAND(c, q_0, 0)`.
2. The edge $c \xrightarrow{\text{ISA}} a$ keeps q_0 . The next edge reaches t_A at depth 2, so t_A is attached directly to q_0 .
3. The edge $c \xrightarrow{r} b$ creates a child context q_1 labelled by r . The edge $b \xrightarrow{\text{ISA}} t_B$ keeps q_1 , so t_B is attached to q_1 .
4. The edge $b \xrightarrow{s} z$ creates a child context q_2 of q_1 . The call reaches z at depth 2; since z is not a token, that branch is marked uncovered in q_2 .

The returned representation is

```
ROOT [q0]
|-- token: t_A
'-- r [q1]
    |-- token: t_B
    '-- s [q2]
        '-- uncovered
```

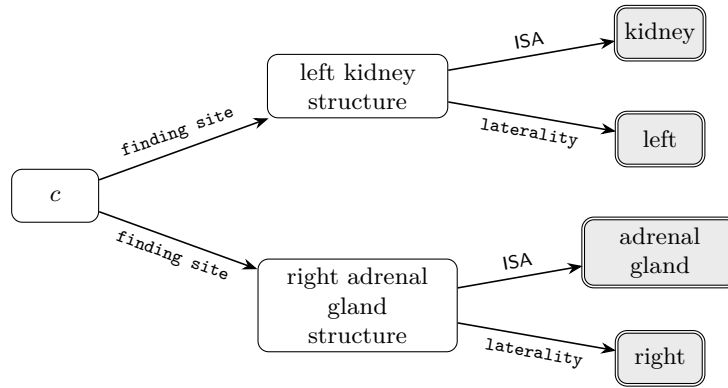
Equivalently, this is the parenthesized expression

$$(t_A, r(t_B, s(\text{uncovered}))).$$

The contexts only retain this parenthesization; the concept vocabulary of the output remains $\mathcal{T}$.

3.3 Why contexts are needed

Consider now a concept c with two site occurrences:



The flat representation

$$\{(\text{finding site}, \text{kidney}), (\text{finding site}, \text{adrenal gland}), (\text{laterality}, \text{left}), (\text{laterality}, \text{right})\}$$

keeps the tokens but loses which laterality belongs to which site. The context tree preserves the two occurrences of `finding site`:

```
ROOT
|-- finding site [context q1]
|   |-- token: kidney
|   '-- laterality [context q2]
|       '-- token: left
```

```

-- finding site [context q3]
  |-- token: adrenal gland
  |-- laterality [context q4]
    |-- token: right

```

Thus, **left** remains inside the context of the kidney site, while **right** remains inside the context of the adrenal gland site.

4 Semantic coverage

The context tree is the final representation once a token set $\mathcal{T}$ has been selected. To select $\mathcal{T}$, we use a semantic coverage score. The score deliberately ignores the context labels: it only asks whether the recursive expansion of a concept reaches tokens within the depth budget D . This gives an efficient objective for greedy selection, while the richer context tree is generated afterwards for the selected tokens.

Definition 1 (Coverage at horizon h). *For $u \in \mathcal{V}$, $\mathcal{T} \subseteq \mathcal{V}$, and $h \in \{0, \dots, D\}$, let*

$$S_h(u, \mathcal{T}) \in [0, 1]$$

denote the semantic coverage of u when at most h additional relationships may be traversed.

The base case is with horizon 0, then the coverage is 1 for tokens and 0 for other concepts:

$$S_0(u, \mathcal{T}) = \mathbf{1}[u \in \mathcal{T}].$$

For $h \geq 1$, we aggregate the coverages from the previous horizon $h - 1$:

$$S_h(u, \mathcal{T}) = \begin{cases} 1, & u \in \mathcal{T}, \\ 0, & u \notin \mathcal{T} \text{ and } d^+(u) = 0, \\ \frac{1}{d^+(u)} \sum_{(u,r,v) \in \text{Out}(u)} S_{h-1}(v, \mathcal{T}), & \text{otherwise.} \end{cases} \quad (1)$$

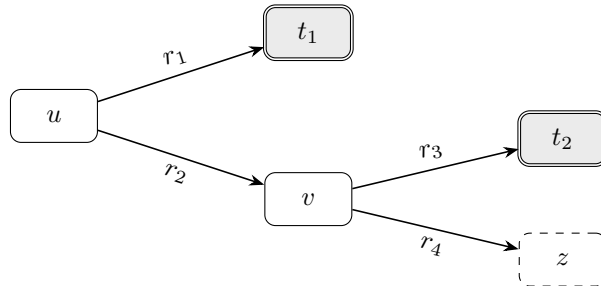
Note that token is absorbing: as soon as $u \in \mathcal{T}$, its coverage is 1 for every remaining depth. Otherwise, the coverage of u is the average coverage of its outgoing relationships.

Finally, the global score of a token set is the mean of the semantic coverage of each mapped concept:

$$F_D(\mathcal{T}) = \frac{1}{|\mathcal{M}|} \sum_{c \in \mathcal{M}} S_D(c, \mathcal{T}). \quad (2)$$

Hence, $F_D(\emptyset) = 0$ and $0 \leq F_D(\mathcal{T}) \leq 1$.

Numerical example. The following toy graph contains two tokens $t_1, t_2 \in \mathcal{T}$ and one uncovered leaf $z \notin \mathcal{T}$:



With $D = 2$, we have $S_1(t_1, \mathcal{T}) = 1$ and

$$S_1(v, \mathcal{T}) = \frac{1}{2} S_0(t_2, \mathcal{T}) + \frac{1}{2} S_0(z, \mathcal{T}) = \frac{1}{2}.$$

Therefore,

$$S_2(u, \mathcal{T}) = \frac{1}{2} S_1(t_1, \mathcal{T}) + \frac{1}{2} S_1(v, \mathcal{T}) = \frac{1}{2} + \frac{1}{4} = \frac{3}{4}.$$

Each branching point therefore splits the weight of the current branch among its outgoing edges. Every relationship has the same weight in this first version, but we can imagine putting different weights on relationships in future work.

5 Greedy selection and submodularity

Once the coverage score is defined, token selection is a repeated choice. Starting from no tokens, we temporarily add each admissible candidate, measure how much the global coverage would increase, keep the candidate with the largest increase, and repeat until k tokens have been selected. This is the greedy selection algorithm.

Define the set of candidates that may have nonzero gain:

$$\mathcal{V}_D = \mathcal{M} \cup \{u \in \mathcal{V} : u \text{ is reachable from a concept in } \mathcal{M} \text{ by at most } D \text{ relationships}\}.$$

We seek

$$\mathcal{T}_k \in \arg \max_{\mathcal{T} \subseteq \mathcal{V}_D, |\mathcal{T}|=k} F_D(\mathcal{T}).$$

For $t \notin \mathcal{T}$, the marginal gain is the increase obtained by adding t to the current set:

$$\Delta_D(t \mid \mathcal{T}) = F_D(\mathcal{T} \cup \{t\}) - F_D(\mathcal{T}). \quad (3)$$

The greedy algorithm constructs $\mathcal{T}_0 = \emptyset$, then

$$t_i \in \arg \max_{t \in \mathcal{V}_D \setminus \mathcal{T}_{i-1}} \Delta_D(t \mid \mathcal{T}_{i-1}), \quad \mathcal{T}_i = \mathcal{T}_{i-1} \cup \{t_i\}. \quad (4)$$

Definition 2 (Submodularity). *A function F is submodular if, for $A \subseteq B$ and $t \notin B$,*

$$F(A \cup \{t\}) - F(A) \geq F(B \cup \{t\}) - F(B).$$

Thus, the marginal contribution of a token cannot increase as more tokens have already been selected.

Proposition 1. *The score F_D is normalized, monotone, and submodular.*

Justification (No need to read). Starting from a concept c , one may sample in advance a walk of maximum length D by choosing uniformly one outgoing edge occurrence at each step. For a fixed walk p , the function

$$\mathcal{T} \mapsto \mathbf{1}[p \cap \mathcal{T} \neq \emptyset]$$

is a coverage function. Recurrence (1) and F_D are positive averages of such functions; they are therefore monotone and submodular.

The classical result of Nemhauser, Wolsey, and Fisher gives the greedy algorithm the guarantee

$$F_D(\mathcal{T}_k^{\text{greedy}}) \geq (1 - 1/e) \max_{|\mathcal{T}| \leq k} F_D(\mathcal{T}) \quad (5)$$

[1]. **Submodularity allows the use of *lazy greedy* without changing the solution returned by exact greedy** [2, 3] (see Section 6.1).

6 Token-selection algorithm

Section 5 specifies the choice to make at each iteration: test the remaining token candidates and select the one with the largest current gain. This section gives the algorithm used to perform that selection efficiently.

Exact greedy recomputes the gain of every remaining token candidate after every selected token. Lazy greedy avoids most of these recomputations. It stores one previously computed gain for each candidate. Thanks to submodularity, this stored value is an upper bound on the candidate's current gain after other tokens have been added. The algorithm repeatedly recomputes only the candidate with the largest stored bound. If its recomputed gain is at least the largest bound still present in the queue, that candidate is necessarily the true maximizer and is selected as the new token. Otherwise, it is returned to the queue with its updated gain.

6.1 Overall lazy-greedy loop

The following algorithm is the complete outer selection algorithm. Procedures `CANDIDATE_DELTA` and `COMMIT` are detailed after. The procedure `CANDIDATE_DELTA` evaluates one candidate for the current set $\mathcal{T}$ and returns both its gain and the sparse changes that would be caused by selecting it. The procedure `COMMIT` applies those changes only after the candidate has been accepted.

Listing 2: Selection of k tokens by lazy greedy

```

SELECT_TOKENS(k):
    T = empty set
    current_score = 0
    S[h,u] = 0 for every h = 0,...,D and every u in V
    Q = empty max-priority queue

    # Initial gains are exact.
    for each candidate t in V_D:
        gain, unused_delta = CANDIDATE_DELTA(t)
        Q.push(candidate=t, bound=gain)

    for i = 1,...,k:
        while not selected:
            t, old_bound = Q.pop_max()
            gain, delta = CANDIDATE_DELTA(t)
            best_other_bound = Q.max_bound()

            if gain >= best_other_bound:
                COMMIT(t, delta)
                current_score = current_score + gain
                record selected token t, gain, and current_score

                selected = true

            else:
                Q.push(candidate=t, bound=gain)

    return T

```

Exact greedy is obtained by recomputing all remaining candidates inside each iteration and selecting the largest gain. Lazy greedy returns the same choice but usually calls `CANDIDATE_DELTA` for far fewer candidates.

6.2 Stored state and complexity

For the current token set $\mathcal{T}$, we store

$$S[h, u] = S_h(u, \mathcal{T}), \quad h = 0, \dots, D, \quad u \in \mathcal{V}.$$

We also store $\text{Out}(u)$, $\text{In}(v)$, the current token set, and the priority queue over $\mathcal{V}_D$. The total memory cost is

$$O(D|\mathcal{V}| + |\mathcal{E}| + |\mathcal{V}_D|) = O(D|\mathcal{V}| + |\mathcal{E}|).$$

For a fixed set $\mathcal{T}$, a full recomputation of (1) costs

$$O(D(|\mathcal{V}| + |\mathcal{E}|)).$$

The local candidate evaluation below avoids this full recomputation.

6.3 Local computation of one candidate gain

Adding a candidate t can only change t itself and vertices that can reach t by following at most D relationships in reverse. We therefore propagate only positive changes through the incoming adjacency lists.

At horizon 0, only t changes. At horizon $h \geq 1$, a change at a destination v contributes its share to every predecessor u through an incoming edge (u, r, v) . Existing tokens remain equal to 1 and are not propagated through. The candidate t is explicitly overridden to 1 at every horizon. At the end, only the changes at horizon D for mapped concepts contribute to the global gain.

Formally, for $t \notin \mathcal{T}$, define

$$\delta_h^t(u) = S_h(u, \mathcal{T} \cup \{t\}) - S_h(u, \mathcal{T}).$$

At level 0:

$$\delta_0^t(u) = \mathbf{1}[u = t](1 - S_0(t, \mathcal{T})). \quad (6)$$

For $h \geq 1$:

$$\delta_h^t(u) = \begin{cases} 1 - S_h(t, \mathcal{T}), & u = t, \\ 0, & u \in \mathcal{T} \text{ or } d^+(u) = 0, \\ \frac{1}{d^+(u)} \sum_{(u,r,v) \in \text{Out}(u)} \delta_{h-1}^t(v), & u \notin \mathcal{T} \cup \{t\}, d^+(u) > 0. \end{cases} \quad (7)$$

The marginal gain is then

$$\Delta_D(t \mid \mathcal{T}) = \frac{1}{|\mathcal{M}|} \sum_{c \in \mathcal{M}} \delta_D^t(c). \quad (8)$$

Listing 3: Local computation of one candidate gain

```
CANDIDATE_DELTA(t):
    delta[0] = {t: 1 - S[0,t]}

    for h = 1,...,D:
        delta[h] = empty dictionary

        for (v, dv) in delta[h-1]:
            for each incoming edge (u, r, v) in In(v):
                if u not in T and u != t:
                    delta[h][u] += dv / out_degree[u]
                    # initialize delta[h][u] to 0 if absent

        override = 1 - S[h,t]
        if override > 0:
            delta[h][t] = override

    gain = sum(delta[D].get(c, 0) for c in M) / |M|
    return gain, delta
```

The test `override > 0` keeps the dictionaries sparse: a zero change is not stored and therefore cannot trigger useless scans of incoming edges at the next horizon.

The affected entries for candidate t are exactly the pairs (h, u) stored in `delta`. Once the best candidate t^* has been selected, its already computed changes are applied without scanning the full array S :

Listing 4: Commit of the selected candidate

```
COMMIT(t_star, delta):
    for h = 0,...,D:
        for (u, du) in delta[h]:
            S[h,u] = S[h,u] + du
    T.add(t_star)
```

The cost of one candidate evaluation is proportional to the number of incoming edge occurrences actually traversed. The worst case remains $O(D|\mathcal{E}|)$, but the intended cost is that of the reverse neighbourhood actually affected by the candidate. Temporary memory is proportional to the number of stored pairs (h, u) .

7 Experiments

Experiments for the method. For $D \in \{1, 2, 3, 4, 5\}$, measure the size of $\mathcal{V}_D$, the number of reachable relationships, and degree distributions in the induced SNOMED graph. Then run greedy up to $k_{\max}$ and record

$$k \mapsto F_D(\mathcal{T}_k),$$

as well as the distribution of $S_D(c, \mathcal{T}_k)$ over $c \in \mathcal{M}$. For several pairs of concepts from (D, k) , generate the context trees and measure their size: number of contexts, tokens used, uncovered branches.

Subsequent evaluation. We may compare this method with the existing distance-based score, random selections, etc. And also with exact match, conciseness, uniqueness, etc.

8 Perspectives

Redundancy and swaps. The greedy algorithm never removes a token. Processing based on marginal loss may remove or replace a token that has become of little value due to other tokens added after it, and that cover its semantics.

Primitive and fully defined concepts. Replacing a primitive concept by the expansion of its outgoing relationships may lose information that is not explicitly represented by those relationships. A possible extension

is therefore to introduce a reconstructibility factor:

$$\alpha(u) = \begin{cases} 1, & \text{if } u \text{ is fully defined,} \\ \eta, & \text{if } u \text{ is primitive,} \end{cases} \quad 0 \leq \eta < 1.$$

The coverage recurrence can then be modified as follows:

$$S_0^{(\eta)}(u, \mathcal{T}) = \mathbf{1}[u \in \mathcal{T}],$$

and, for $h \geq 1$,

$$S_h^{(\eta)}(u, \mathcal{T}) = \begin{cases} 1, & u \in \mathcal{T}, \\ 0, & u \notin \mathcal{T} \text{ and } \text{Out}(u) = \emptyset, \\ \alpha(u) \frac{1}{|\text{Out}(u)|} \sum_{(u,r,v) \in \text{Out}(u)} S_{h-1}^{(\eta)}(v, \mathcal{T}), & \text{otherwise.} \end{cases}$$

A primitive concept that is itself selected as a token still has coverage 1, because it is preserved atomically rather than reconstructed from its relationships. The penalty applies when a non-token concept is expanded.

Ontology-based similarity. A similarity measure developed independently [6] may be used to evaluate preservation of semantic neighbourhoods and the severity of collisions. It may later be added to the marginal gain, for example as a facility-location term, provided that the properties of the resulting score are reassessed [4, 5].

References

- [1] G. L. Nemhauser, L. A. Wolsey, and M. L. Fisher. An analysis of approximations for maximizing submodular set functions—I. *Mathematical Programming*, 14:265–294, 1978. doi: 10.1007/BF01588971.
- [2] M. Minoux. Accelerated greedy algorithms for maximizing submodular set functions. In *Optimization Techniques*, Lecture Notes in Control and Information Sciences, vol. 7, pp. 234–243. Springer, 1978. doi: 10.1007/BFb0006528.
- [3] J. Leskovec, A. Krause, C. Guestrin, C. Faloutsos, J. VanBriesen, and N. Glance. Cost-effective outbreak detection in networks. In *Proceedings of KDD*, pp. 420–429, 2007. doi: 10.1145/1281192.1281239.
- [4] T. Pedersen, S. V. S. Pakhomov, S. Patwardhan, and C. G. Chute. Measures of semantic similarity and relatedness in the biomedical domain. *Journal of Biomedical Informatics*, 40(3):288–299, 2007. doi: 10.1016/j.jbi.2006.06.004.
- [5] D. Chang. A vector-based semantic relatedness measure using multiple relations within SNOMED CT and UMLS. *Journal of Biomedical Informatics*, 2022. doi: 10.1016/j.jbi.2022.104118.
- [6] A. Baudin, C. Gaudet-Blavignac, C. Lovis, and M. Bjelogrić. Explainable framework for ontology-based similarity: a use case on SNOMED CT. *Studies in Health Technology and Informatics*, vol. 336, pp. 443–447, 2026. doi: 10.3233/SHTI260194. PMID: 42174870.